



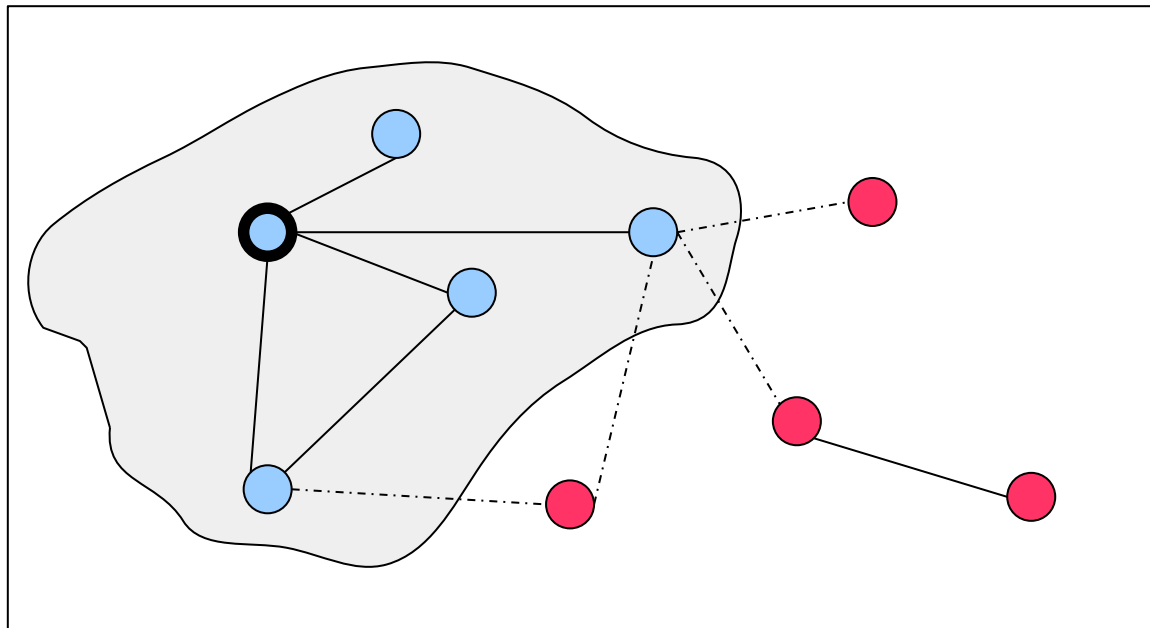
Criptografía y Seguridad

Políticas de seguridad y
Control de acceso

Definiendo seguridad en aplicaciones

No hay una definición única de seguridad.

En un contexto (aplicación, servicio, empresa, etc) la política de seguridad define los modos y usos que se consideran seguros.



Un sistema es seguro mientras sus estados queden dentro de la política de seguridad

Tríada

Tríada CIA

Confidencialidad
Integridad
Disponibilidad





Criptografía y Seguridad

Conceptos de control de acceso

Conceptos

Sujeto

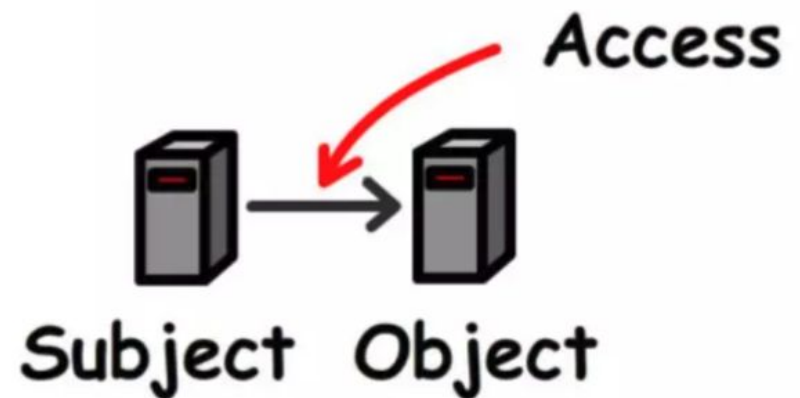
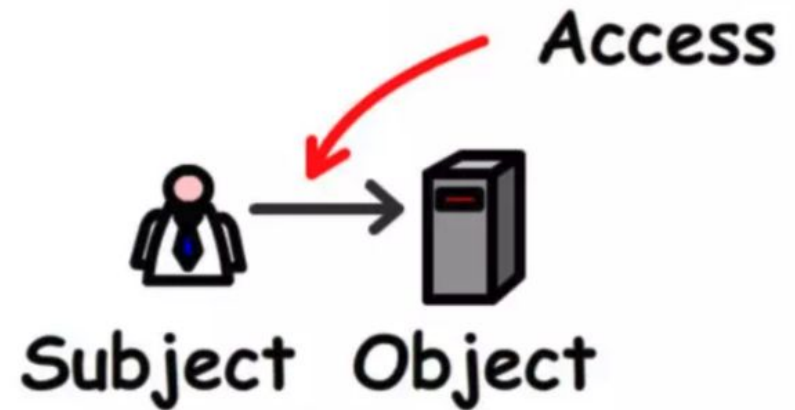
Persona o servicio que trata de acceder a un Objeto

Objeto

Recurso (Dato, Servicio) que puede ser accedido para ser leído, modificado o accionado por un sujeto.

Acceso

Formas de interacción entre un sujeto y los objetos.



Ejemplo: Linux

Sujeto

Objeto

Acceso

```
sara@sara-pnap:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```

Ejemplo: Linux

Sujeto

Objeto

Acceso

```
jpp@jpp:/boot$ ls -la
total 39132
drwxr-xr-x  3 root root    4096 2011-05-13 08:52 .
drwxr-xr-x 23 root root    4096 2011-05-04 09:27 ..
-rw-r--r--  1 root root 700761 2011-03-18 16:33 abi-2.6.35-28-generic
-rw-r--r--  1 root root 730039 2011-04-11 01:24 abi-2.6.38-8-generic
-rw-r--r--  1 root root 122616 2011-03-18 16:33 config-2.6.35-28-generic
-rw-r--r--  1 root root 130313 2011-04-11 01:24 config-2.6.38-8-generic
drwxr-xr-x  3 root root    12288 2011-05-04 09:32 grub
-rw-r--r--  1 root root 11008098 2011-04-15 08:58 initrd.img-2.6.35-28-generic
-rw-r--r--  1 root root 13134896 2011-05-13 08:52 initrd.img-2.6.38-8-generic
-rw-r--r--  1 root root   160988 2010-10-22 09:08 memtest86+.bin
-rw-r--r--  1 root root   163168 2010-10-22 09:08 memtest86+_multiboot.bin
-rw-r--r--  1 root root 2344143 2011-03-18 16:33 System.map-2.6.35-28-generic
-rw-----  1 root root 2654256 2011-04-11 01:24 System.map-2.6.38-8-generic
-rw-r--r--  1 root root    1336 2011-03-18 16:35 vmcoreinfo-2.6.35-28-generic
-rw-----  1 root root    1368 2011-04-11 01:26 vmcoreinfo-2.6.38-8-generic
-rw-r--r--  1 root root 4342384 2011-03-18 16:33 vmlinuz-2.6.35-28-generic
-rw-----  1 root root 4523936 2011-04-11 01:24 vmlinuz-2.6.38-8-generic
jpp@jpp:/boot$
```

La triada: punto de vista de accesos

Las políticas de seguridad se implementan controlando como los sujetos acceden a los objetos.

Confidencialidad:

Existen sujetos que NO puede acceder a ciertos objetos

Integridad:

Existen objetos que solo pueden ser modificados por ciertos sujetos confiables

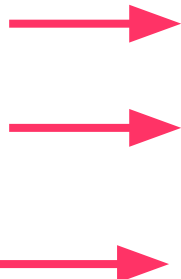
Disponibilidad:

Existen objetos que pueden ser accedidos por ciertos sujetos cuando lo requieran

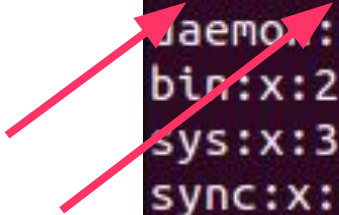
Identidad del Sujeto

Lo que identifica unívocamente al sujeto dentro del universo de posibles sujetos.

Es un requerimiento para la autenticación.



```
DistinguishedName : CN=Administrator,CN=Users,DC=automationlab,DC=local
Enabled           : True
GivenName         :
Name              : Administrator
ObjectClass       : user
ObjectGUID        : 7d4558a4-cbff-4d5d-bc03-e9566eb4b03b
SamAccountName    : Administrator
SID               : S-1-5-21-1578757976-2349072586-3307742921-500
Surname           :
UserPrincipalName :
```



```
sara@sara-pnap:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```



Criptografía y Seguridad

Modelos de Seguridad

¿Qué es un Modelo de Seguridad?

En un modelo formal lógico.

Define el reglas e interacciones que implementan mecanismos de control de acceso.

Está especializado: Hay modelos centrados en confidencialidad, integridad e incluso disponibilidad.

Evita reinventar la rueda, pero no alcanza por sí solo en un sistema real.

Modelo Bell-LaPadula

- Modelo diseñado para el ámbito militar, focalizado en proteger la **confidencialidad**.
- Define que **la información debe clasificarse** de acuerdo al nivel de confidencialidad de la misma.
- Todos los sujetos deben tener un **nivel de acceso** definido, con los mismos posibles niveles de la información.

Top Secret

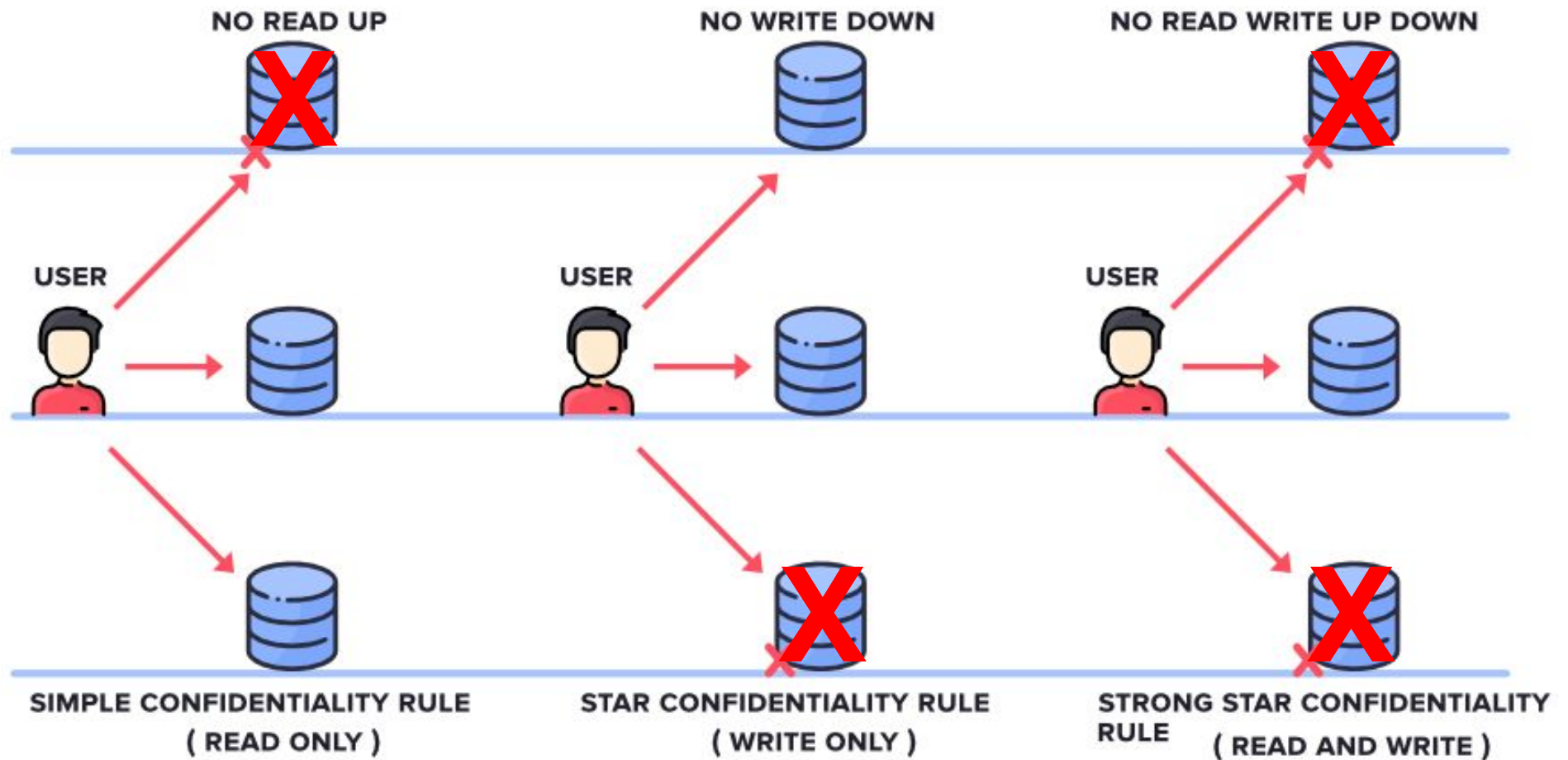
Secret

Confidential

Unclassified

Modelo Bell-La Padula

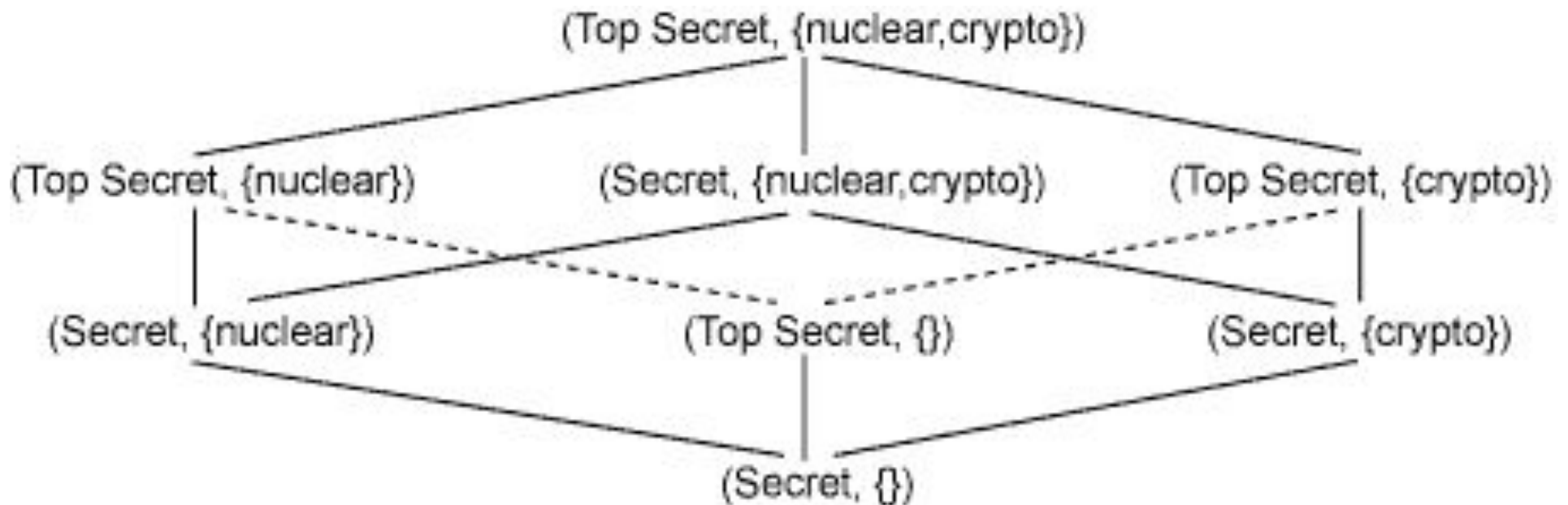
BELL - LAPADULA MODEL



Read down, Write up

Modelo Bell-La Padula (Lattice)

- El modelo agrega etiquetas para implementar el concepto de need-to-know.
- Si un general se encarga de criptografía, no tiene por qué conocer los secretos de los misiles, sin importar su nivel de acceso.



Modelo Biba

- Creado por Kenneth J Biba.
- Modelo diseñado para proteger la **integridad de la información**.
- Define que **la información debe clasificarse** de acuerdo al nivel de integridad de la misma.
- Todos los sujetos deben tener un **nivel de integridad** definido, con los mismos posibles niveles de la información.

Top Secret

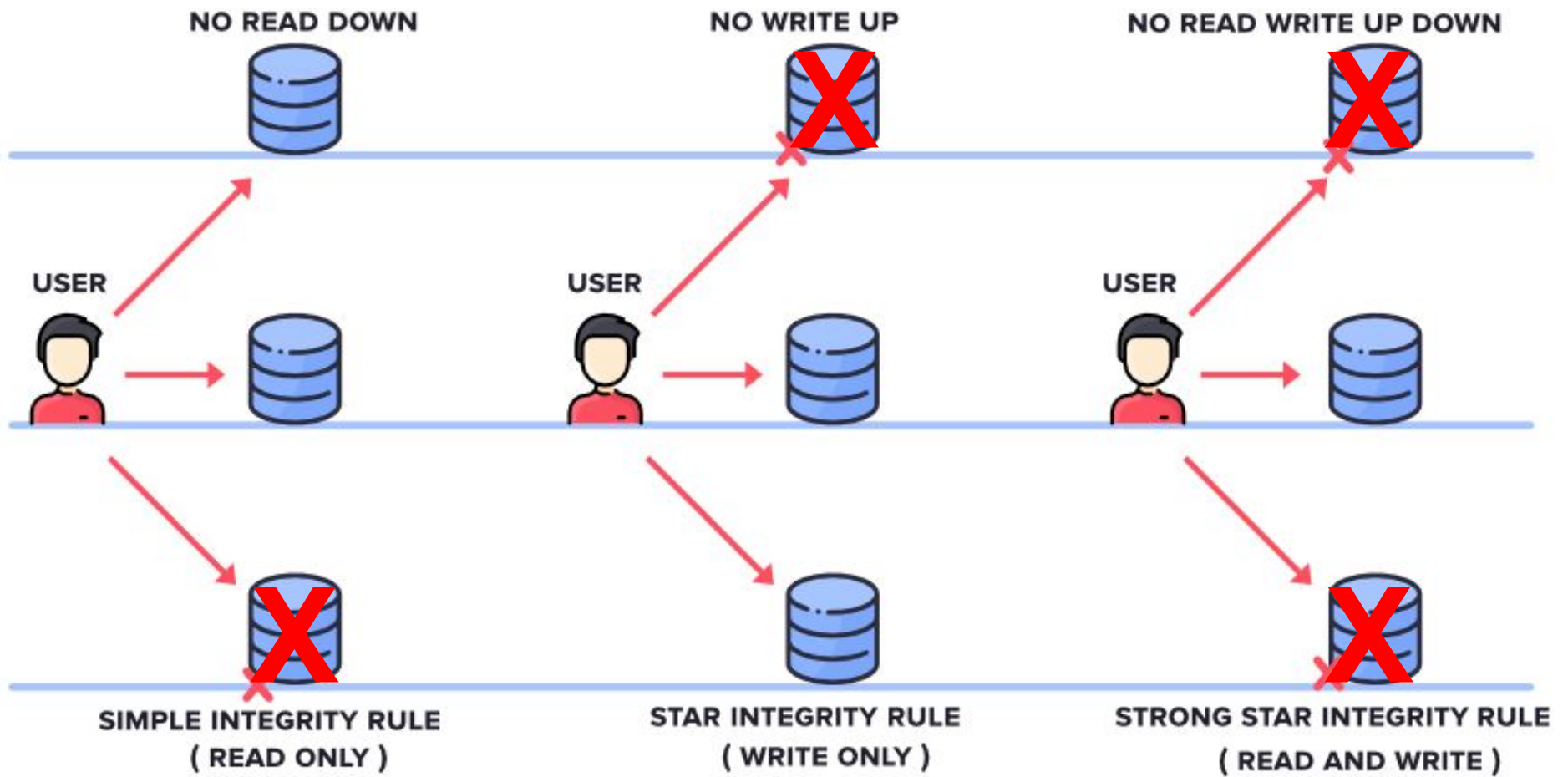
Secret

Confidential

Unclassified

Modelo Biba

BIBA MODEL

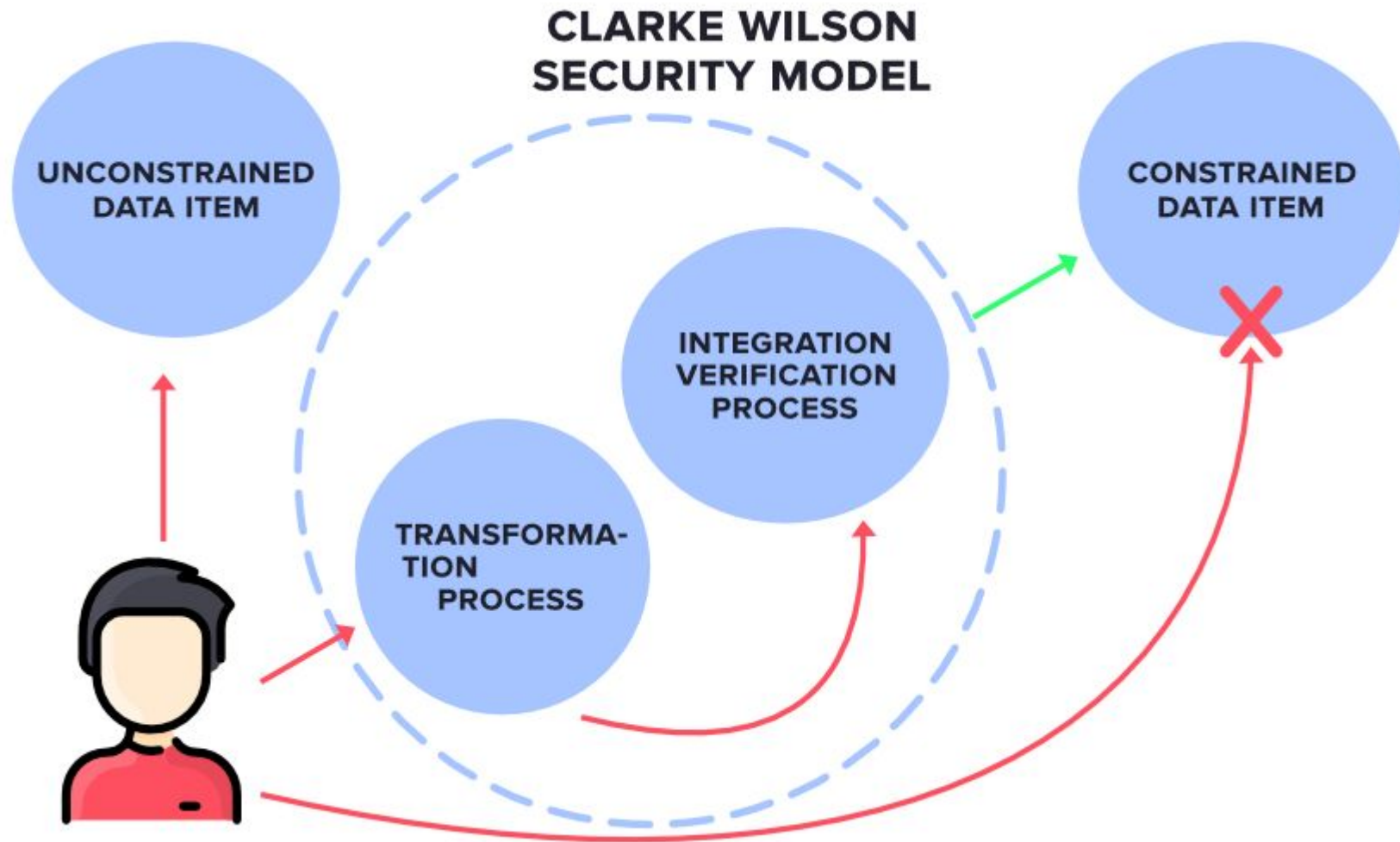


Read up, Write down

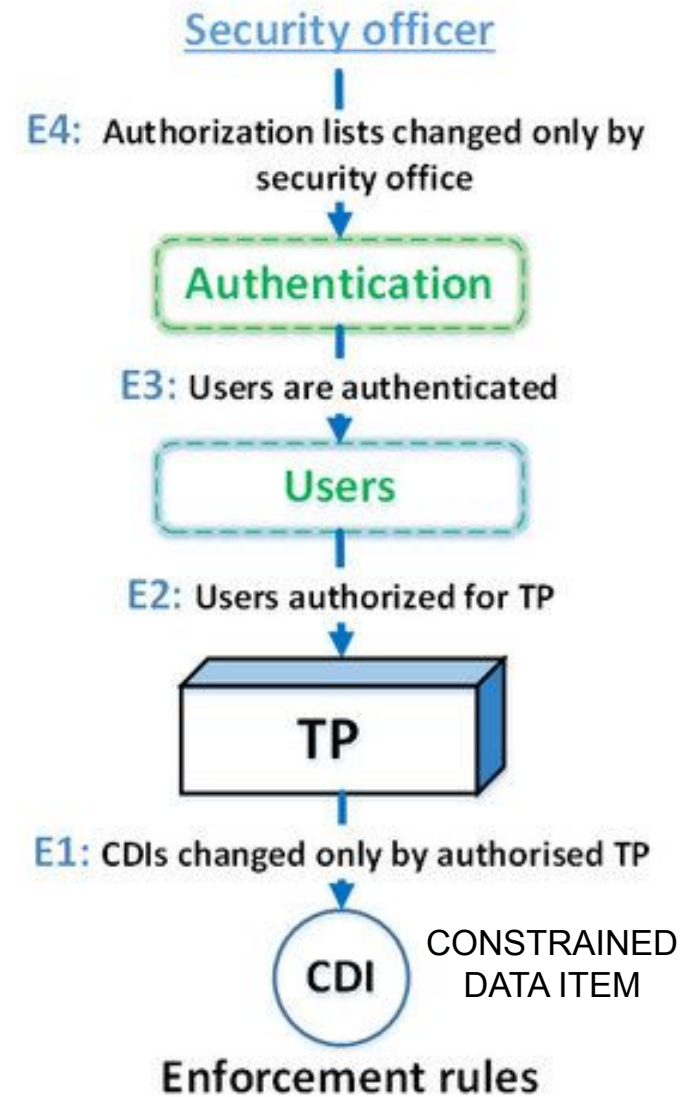
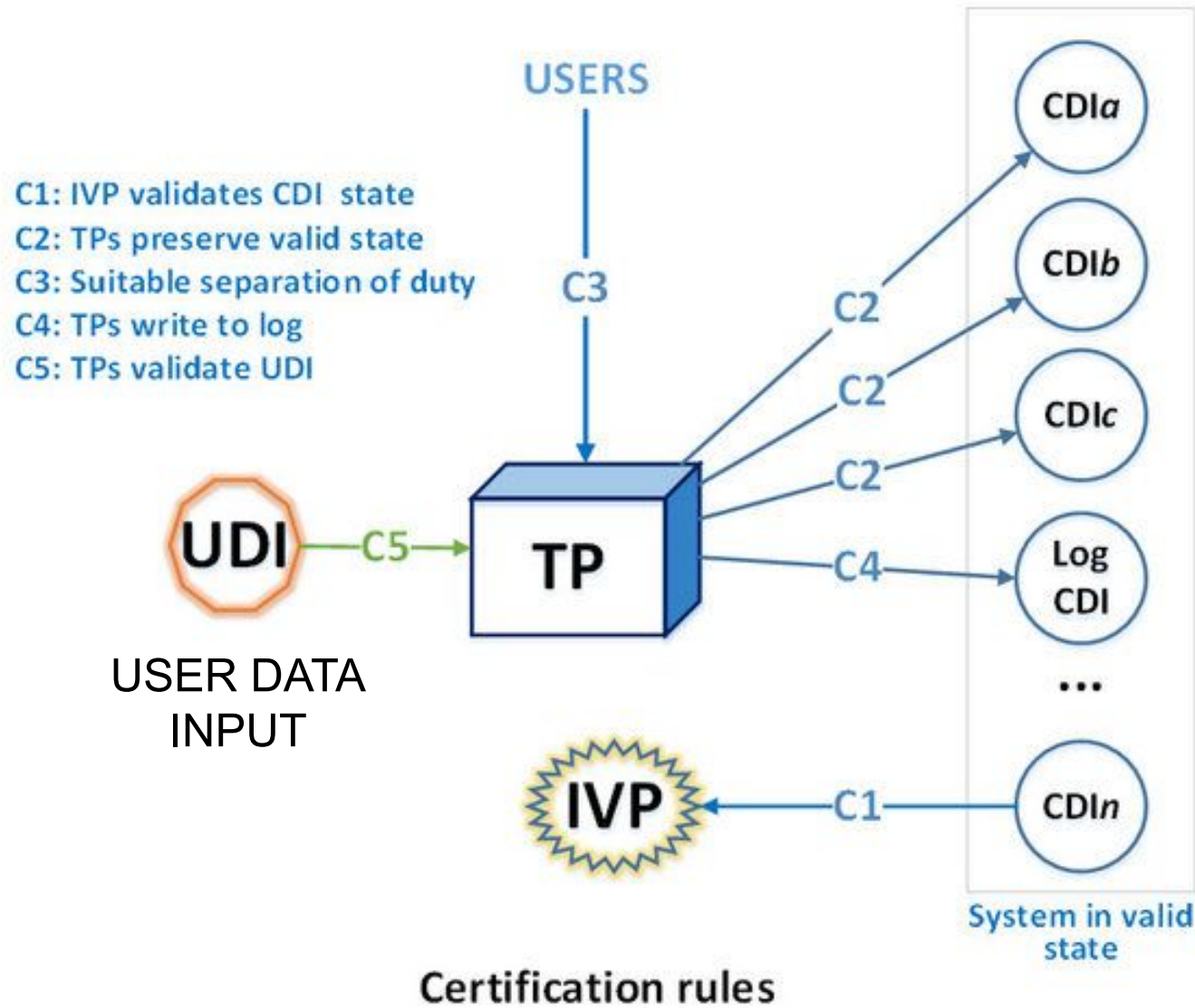
Modelo Clark-Wilson

- Creado por David D. Clark y David R. Wilson.
- Modelo diseñado para proteger la **integridad de la información**.
- Define que los datos pueden estar **restringidos** (protegidos) o sin restricción.
- Los datos restringidos sólo pueden ser modificados por medio de un **proceso de transformación (TP)**.
- Es responsabilidad de los procesos de transformación **validar la integridad** de la información.
- Los sujetos sólo modifican los datos restringidos por medio de los procesos de transformación.
- Incorpora el concepto de **separation of duty**. Quién modifica los datos no puede ser el mismo que verifica la integridad.

Modelo Clark-Wilson



Modelo Clark-Wilson (Formal)



Más allá de los modelos teóricos

- Los modelos vistos hasta ahora son modelo teóricos.
- En la práctica no se pueden aplicar directamente pero son la base de muchas implementaciones.
- La organización por compartimentos de Bell Lapadula aparece en sistemas que manejan información médica.
- Los sistemas de clasificación y archivo de información para medios de prensa, legajos judiciales o declaraciones fiscales implementan controles basados en modelos Biba.
- El concepto de kernel space/user space de muchos sistemas operativos surge del modelo Clark-Wilson.



Criptografía y Seguridad

Control de acceso

Control de Acceso

- El control de acceso es un mecanismo de seguridad que regula quién puede acceder a qué recursos de la aplicación y bajo qué condiciones.

```
struct group_info init_groups = { .usage = ATOMIC_INIT(2) };
struct group_info *groups_alloc(int gidsetsize)
{
    struct group_info *group_info;
    int nblocks;
    int i;

    nblocks = (gidsetsize + NGROUPS_PER_BLOCK - 1) / NGROUPS_PER_BLOCK;
    /* Make sure we always allocate at least one indirect block pointer */
    nblocks = nblocks ? : 1;
    group_info = kmalloc(sizeof(*group_info) + nblocks*sizeof(gid_t *), GFP_USER);
    if
    {
        gro
        gro
        ato

        if (gidsetsize <= NGROUPS_SMALL)
            group_info->nblocks[0] = group_info->small_block;
        else {
            for (i = 0; i < nblocks; i++) {
                gid_t *b;
                b = (void *)__get_free_page(GFP_USER);
                if (!b)
                    goto out_undo_partial_alloc;
                group_info->nblocks[i] = b;
            }
        }
    }
}
```



Mandatory Access Control (MAC)

Se denomina MAC a un control que se aplica en forma obligatoria a todos los accesos de los sujetos a los objetos.

Por ejemplo, puede basarse en un modelo Bell LaPadula Lattice o similar.

El control se aplica sobre:

- todos los accesos
- cambios en las reglas de control, o atributos de estas controlan
- el intercambio de información entre sujetos

Discretionary Access Control (DAC)

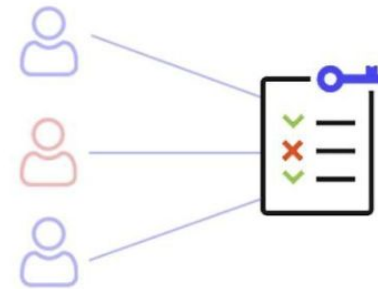
- A diferencia del MAC, la asignación y modificación de permisos queda a discreción del sujeto responsable del objeto o un administrador.
- Es el típico control que se aplica en los repositorios donde los sujetos crean los objetos.
 - File systems
 - Object storage
- Permite distribuir la responsabilidad a **discreción** de los usuarios

Control por extensión

- Lista de Control de Acceso:
(ACL, por sus siglas en inglés)
es una lista de reglas que se utiliza para controlar los permisos de acceso a recursos específicos en un sistema de información.
- Estas reglas especifican qué usuarios o sistemas tienen permisos de acceso y qué acciones pueden realizar sobre los objetos.

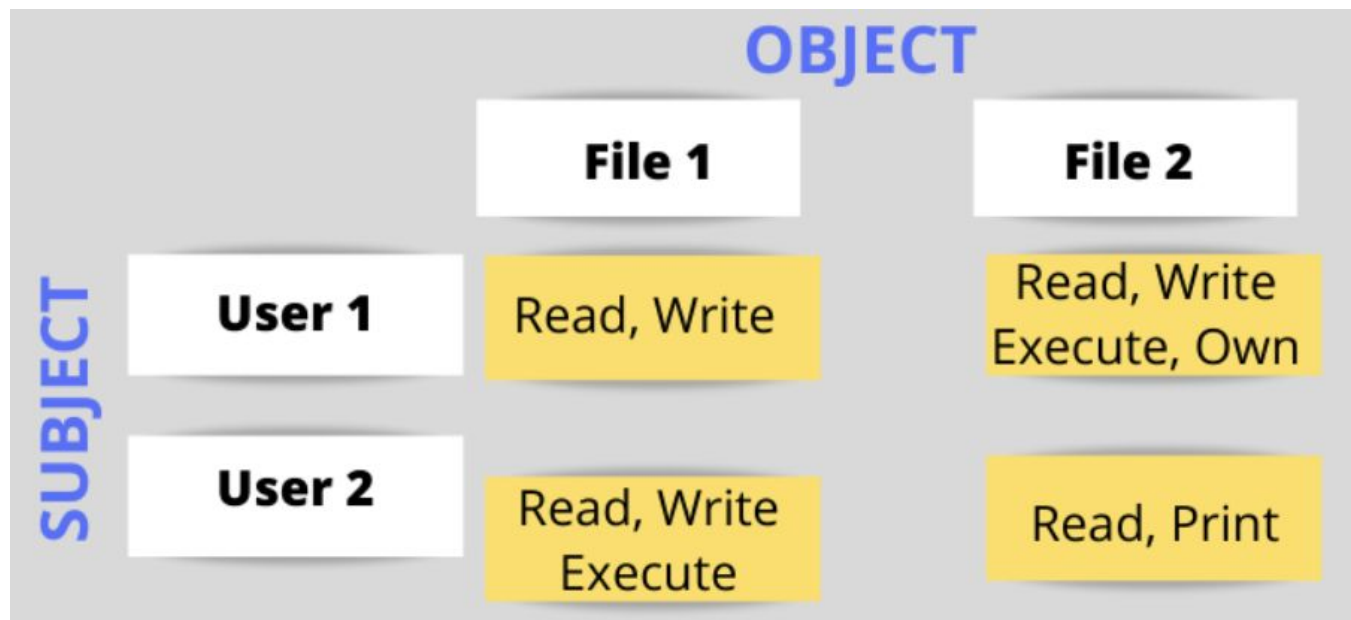
DAC

Access Control List



Control por extensión

- Matriz de accesos
 - Matriz centralizada con todas las combinaciones de sujetos (filas) y objetos (columnas).
 - En cada celda se especifican las acciones que el sujeto en esa fila puede realizar sobre el objeto en esa columna.



The diagram illustrates an Access Control Matrix (ACM) with the following structure:

		OBJECT	
		File 1	File 2
SUBJECT	User 1	Read, Write	Read, Write Execute, Own
	User 2	Read, Write Execute	Read, Print

Control por extensión

- Matriz de accesos
 - Aunque es simple para gestionar permisos, también presenta varios inconvenientes:
 - Escalabilidad: puede volverse extremadamente grande y difícil de manejar.
 - Eficiencia: La búsqueda puede consumir mucho tiempo y recursos computacionales.
 - Mantenimiento: Actualizar permisos puede ser complejo y propenso a errores.
 - Espacio de almacenamiento
 - Limitaciones Dinámicas: La matriz no se adapta bien a entornos dinámicos donde cambian los permisos, los objetos o los permisos.

Control por extensión

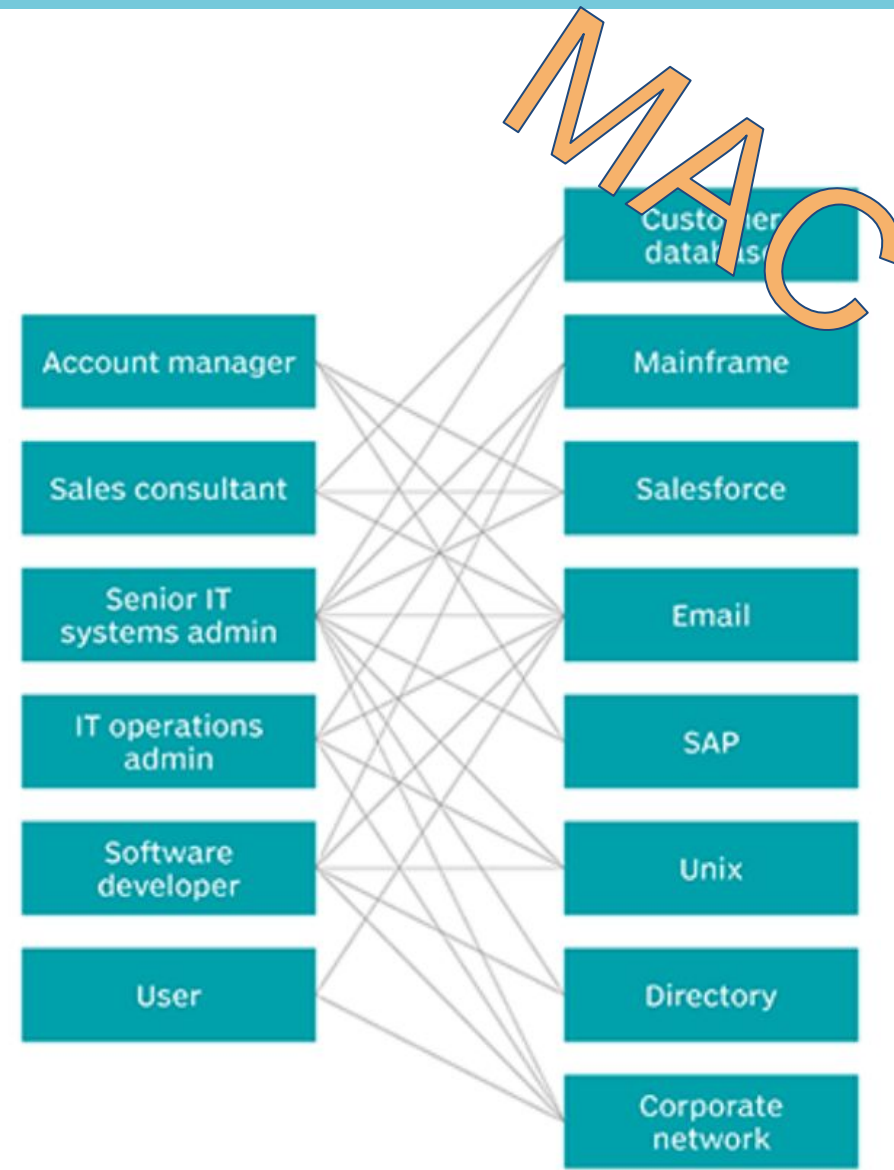
- Matriz de accesos
 - Usualmente se incluyen permisos por defecto, a fin de reducir el tamaño de la matriz.
 - Se pueden usar patrones para identificar los objetos
 - /users/*
 - /home/**/* .tmp

Control por comprensión

- Se define por medio de reglas descentralizadas como los principales pueden accionar sobre los objetos.
- Pueden definir las acciones permitidas o las denegadas.
- Las reglas pueden almacenarse en forma centralizada o estar definidas junto con los objetos.
- El criterio de agrupación define el mecanismo
 - Por roles - RBAC
 - Por atributos - ABAC
 - Por reglas - RuBAC

Control por comprensión - RBAC (roles)

- Role-based access control (RBAC):
 - Los permisos se definen solamente por roles.
 - Los roles se asignan a los sujetos o a grupos.
 - Usualmente los roles tiene una representación funcional de la organización.
 - Gestor de pacientes
 - Analista de compras
 - DBA



Control por comprensión - RuBAC (reglas)

- Rule-based access control (RuBAC):
 - Se basa en una serie de reglas de acceso, que es su forma más básica contiene:
 - Principal
 - Objecto
 - Tipo de Acceso
 - Permitido o bloqueado
 - Puede tener muchas reglas que apliquen. Dependiendo de la implementación, se aplica alguna política de resolución.

MAC

Resolución de múltiples reglas

- Cuando un sistema de reglas posee muchas reglas que aplican en forma contradictoria se utiliza alguna de las siguientes formas de resolución:
 - First match: Las reglas poseen un orden, y la primera que hace match se aplica
 - Best match: La regla más específica es la que aplica.
 - Deny over Allow: Cualquier reglas que deniegue explícitamente algo tiene precedencia sobre las reglas que lo permiten

Resolución de múltiples reglas

- First match

10.1.1.1 => 20.1.1.1:80

No.	Protocol	Source IP	Destination IP	Dest. Port	Action
1	TCP	10.1.1.1	20.1.1.1	80	Accept
2	TCP	10.1.1.2	20.1.1.1	80	Deny
3	TCP	10.1.1.0/24	20.1.1.1	80	Deny
4	TCP	10.1.1.3	20.1.1.1	80	Accept
5	TCP	10.2.2.0/24	20.2.2.5	80	Deny
6	TCP	10.2.2.5	20.2.2.0/24	80	Deny
7	TCP	10.3.3.0/24	20.3.3.9	80	Accept
8	TCP	10.3.3.9	20.3.3.0/24	80	Deny
9	IP	0.0.0.0/0	0.0.0.0/0	0-65535	Deny

Colisión

Orden

Resolución de múltiples reglas

- Best match

10.3.3.9 => 20.3.3.9:80

Protocol	Source IP	Destination IP	Dest. Port	Action
TCP	10.1.1.1	20.1.1.1	80	Accept
TCP	10.1.1.2	20.1.1.1	80	Deny
TCP	10.1.1.0/24	20.1.1.1	80	Deny
TCP	10.1.1.3	20.1.1.1	80	Accept
TCP	10.2.2.0/24	20.2.2.5	80	Deny
TCP	10.2.2.5	20.2.2.0/24	80	Deny
TCP	10.3.3.0/24	20.3.3.9	80	Accept
TCP	10.3.3.9	20.3.3.9	80	Deny
IP	0.0.0.0/0	0.0.0.0/0	0-65535	Deny

Resolución de múltiples reglas

- Deny over Allow

10.1.1.1 => 20.1.1.1:80

Protocol	Source IP	Destination IP	Dest. Port	Action
TCP	10.1.1.1	20.1.1.1	80	Accept
TCP	10.1.1.2	20.1.1.1	80	Deny
TCP	10.1.1.0/24	20.1.1.1	80	Deny
TCP	10.1.1.3	20.1.1.1	80	Accept
TCP	10.2.2.0/24	20.2.2.5	80	Deny
TCP	10.2.2.5	20.2.2.0/24	80	Deny
TCP	10.3.3.0/24	20.3.3.9	80	Accept
TCP	10.3.3.9	20.3.3.0/24	80	Deny
IP	0.0.0.0/0	0.0.0.0/0	0-65535	Deny

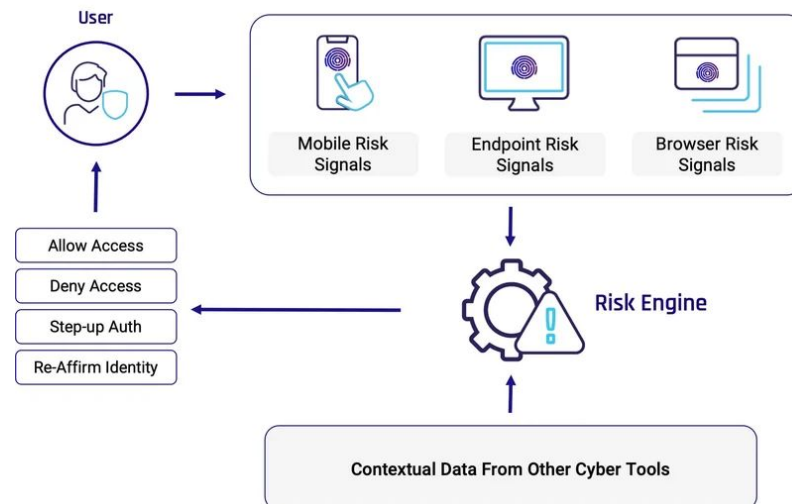
Colisión

Reglas basadas en contexto

- El contexto del acceso lo representan todos los datos asociados al sujeto, el objeto, el momento o lugar del acceso.
- Son elementos que pueden cambiar independientemente de las reglas de acceso.
 - Momento del día
 - Ubicación geográfica del origen de la conexión
 - Departamento al que pertenece la persona que trata de acceder
 - Si el usuario estaba de vacaciones
 - Nivel de confiabilidad de la máquina del DBA.

Reglas basadas en contexto

- Permite definir reglas lógicas para detectar situaciones anómalas. Ejemplo:
 - Un usuario se loguea desde China y desde Argentina con 5 minutos de diferencia.
 - Una usuaria que figura de licencia por maternidad se loguea en una base de datos en el datacenter.
 - Un usuario inicia dos VPNs en menos de 5 minutos con dos IPs distintas.

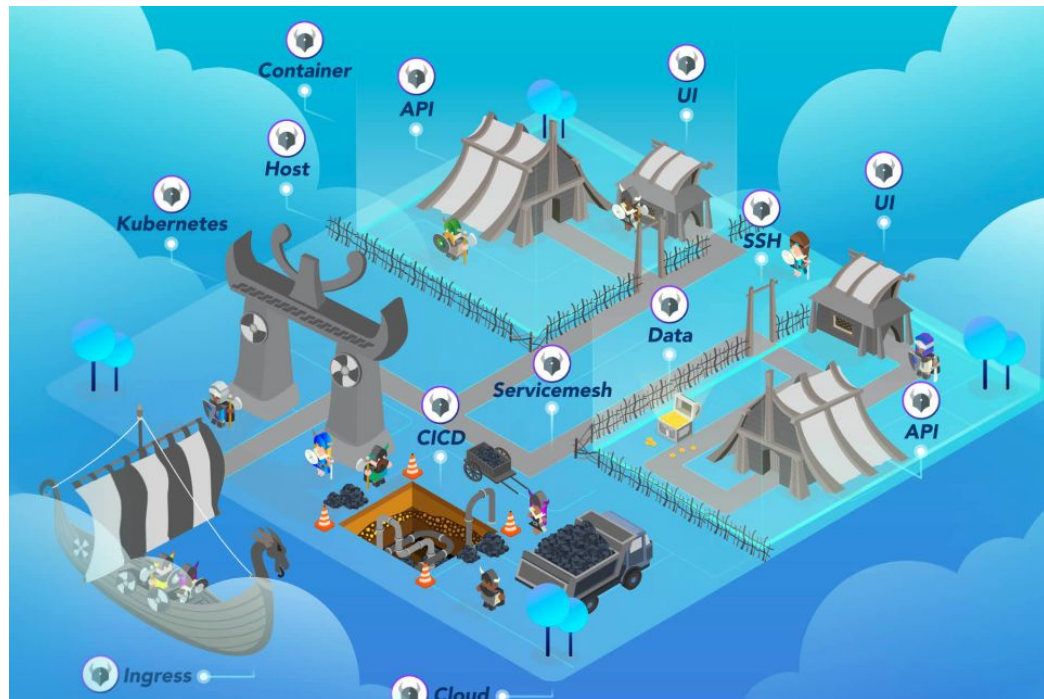


Reglas basadas en contexto

- La complejidad de este tipo de reglas, es que su implementación muchas veces depende de una lógica que se deben programar.
- En varios sistemas no se puede ni implementar esta lógica, ya que no podemos modificarlos.
- En el modelo Zero-Trust, se define una entidad externa al Policy Engine, para evaluar todas las condiciones de contexto.

Open Policy Agent

- Open Policy Agent (OPA) es un framework standard que unifica por medio de un lenguaje programable, la implementación de políticas de decisión.
- Desacopla la implementación de políticas de control de acceso del sistemas del sistema de control.



OPA Policy

- Se escriben en un lenguaje denominado Rego
- Rego proporciona una forma flexible y potente de definir políticas para diversos casos de uso.
- Combinando reglas y aprovechando operadores y funciones integradas, puedes crear políticas comprensivas para imponer restricciones y permisos dentro de las aplicaciones e infraestructura.

Ejemplo de Rego en Kubernetes

```
package kubernetes.admission

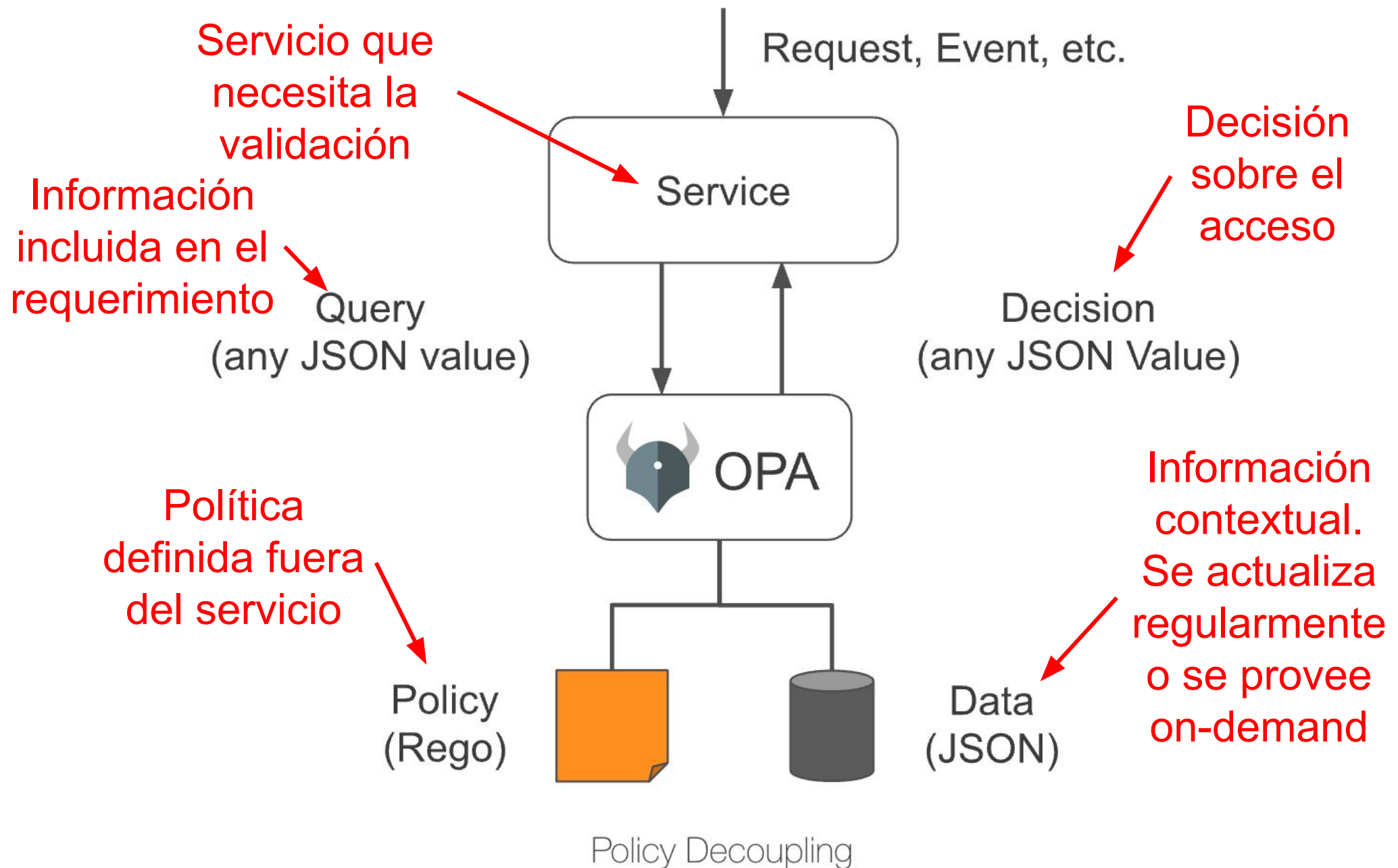
import rego.v1

deny contains reason if {
    some container
    input_containers[container]
    not startswith(container.image, "hooli.com/")
    reason := "container image refers to illegal registry (must be hooli.com)"
}

input_containers contains container if {
    container := input.request.object.spec.containers[_]
}

input_containers contains container if {
    container := input.request.object.spec.template.spec.containers[_]
}
```

Open Policy Agent



OPA Policy - Ejemplo

```
{
  "creditor_account":11111,
  "creditor":"alice",
  "debtor_account":22222,
  "debtor":"bob",
  "period":30,
  "amount":1000
}
```

```
default allow := false

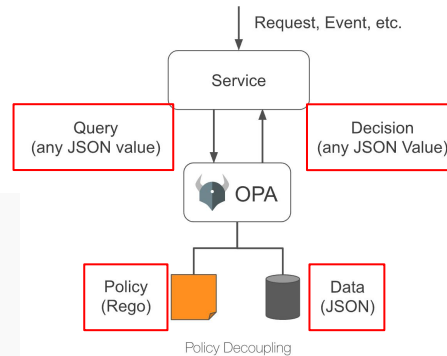
allow if {
  creditor_is_valid
  debtor_is_valid
  period_is_valid
  amount_is_valid
}

creditor_is_valid if
  data.account_attributes[input.creditor_account].owner == input.creditor

debtor_is_valid if
  data.account_attributes[input.debtor_account].owner == input.debtor

period_is_valid if
  input.period <= 30

amount_is_valid if
  data.account_attributes[input.debtor_account].amount >= input.amount
```



```
{
  "allow":true,
  "amount_is_valid":true,
  "creditor_is_valid":true,
  "debtor_is_valid":true,
  "period_is_valid":true
}
```

```
{
  "account_attributes":{
    "11111":{
      "owner":"alice",
      "amount":10000
    },
    "22222":{
      "owner":"bob",
      "amount":10000
    },
    "33333":{
      "owner":"cam",
      "amount":10000
    }
  }
}
```

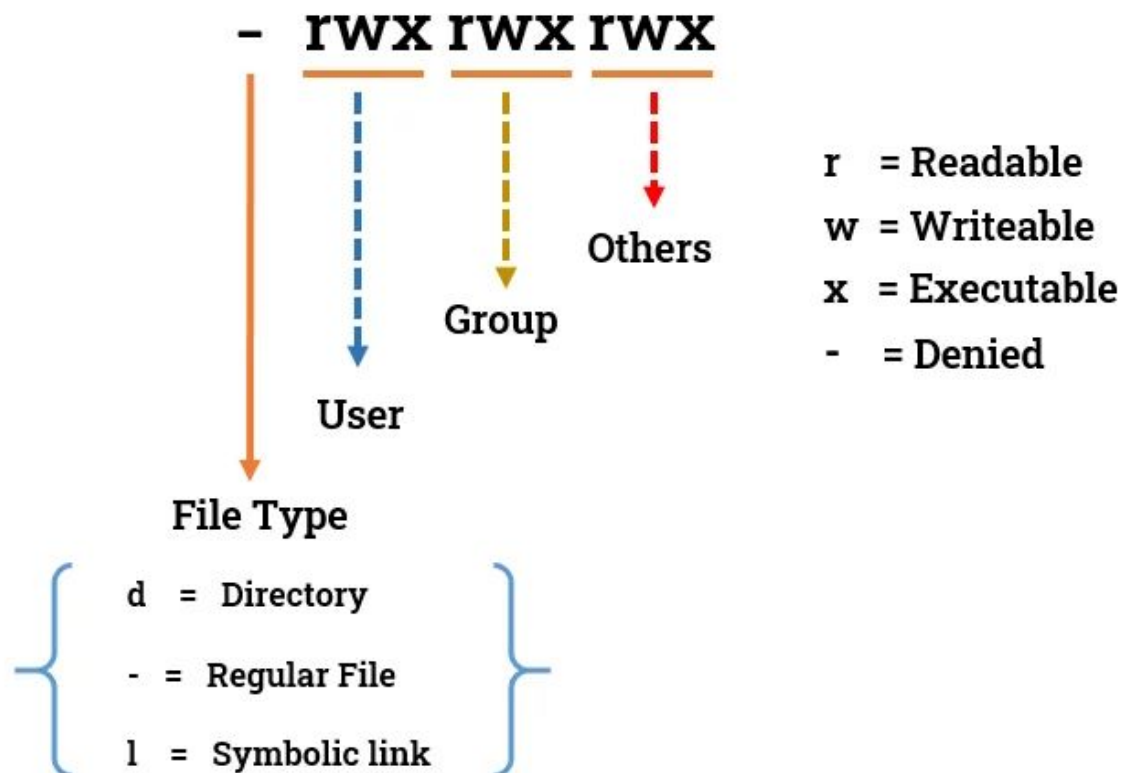


Criptografía y Seguridad

Ejemplos de Control de Acceso

Unix File System

```
→ devel ls -l
total 0
drwxr-xr-x 12 fede staff 384 Sep 30 20:39 allurion
drwxr-xr-x  6 fede staff 192 Jan 20 2023 fclabs
drwxr-xr-x  4 fede staff 128 Oct 27 2022 fede10able
drwxr-xr-x 19 fede staff 608 Nov 18 2023 itba-fcastaneda
-rw-r--r--  1 fede staff   0 Oct  3 09:03 some
→ devel
```



```
→ devel umask -S
u=rwx,g=rx,o=rx
```


SELinux

- Es un control de seguridad adicional al DAC aplicado en Linux por default.
- Se basa en reglas definidas para responder la pregunta:
 - Puede el sujeto acceder al objeto?
- Los procesos poseen un SELinux context
- Todos los recursos que un proceso puede acceder también tienen un contexto definido
 - /var/tmp: t_var_tmp
 - Port 80: t_port_www
 - /var/www/html: t_web_html

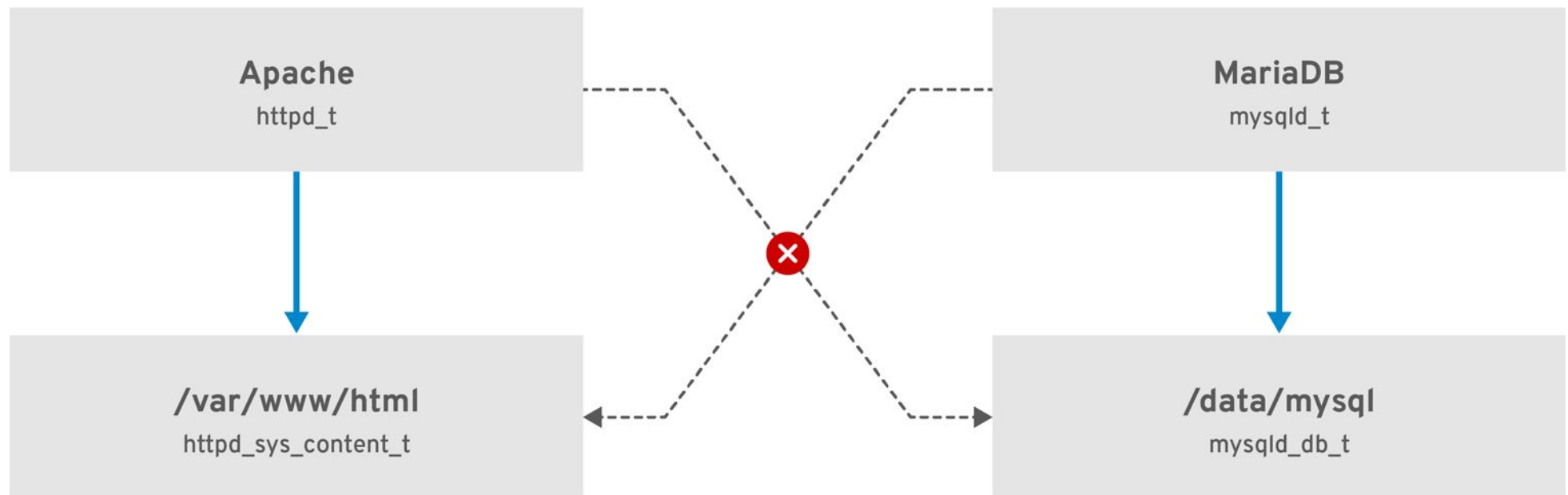
SELinux

- Se utilizan macros para definir los permisos más comunes. El resultado del macro es un conjunto de reglas.
- Ejemplo

```
# macro-expander "mysql_read_config(httpd_t) "  
allow httpd_t mysqld_etc_t:dir { getattr search open read lock ioctl };  
allow httpd_t mysqld_etc_t:file { open { getattr read ioctl lock } };  
allow httpd_t mysqld_etc_t:lnk_file { getattr read };
```

SELinux

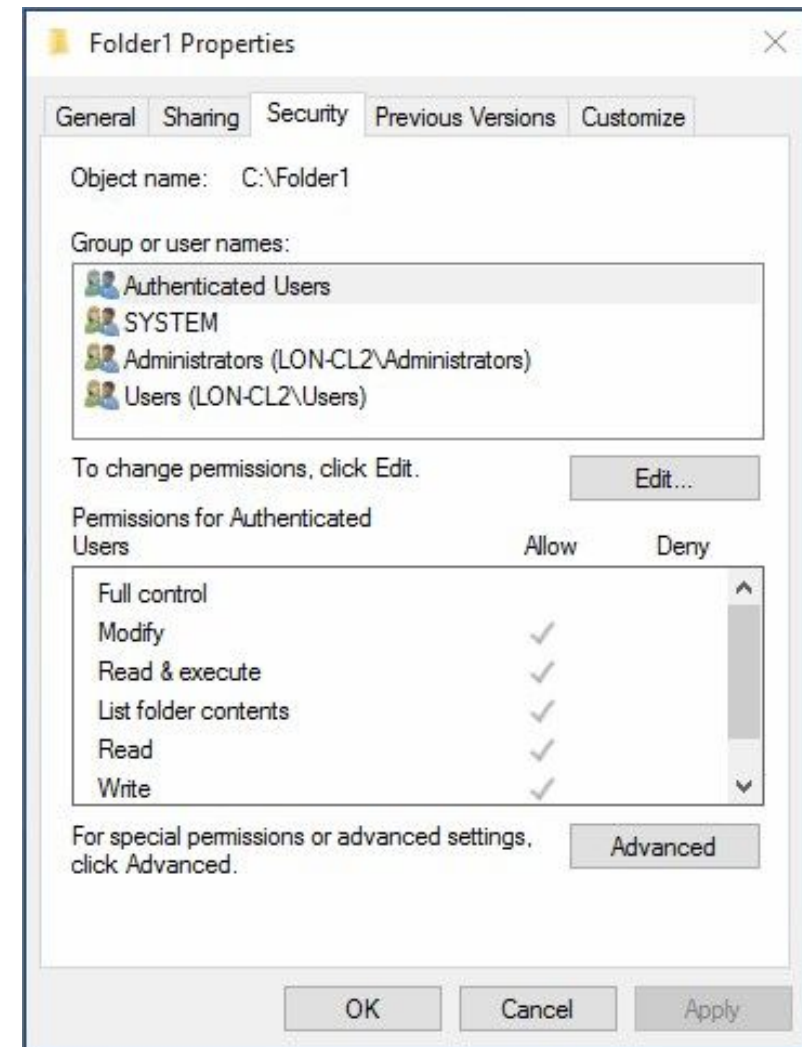
- Dos procesos corriendo con UID 0 pueden tener restringidos los accesos a los recursos del otro gracias a SELinux.



RHEL_467048_0218

Windows File System

- El dueño de cada archivo o directorio define los permisos (DAC).
- Los sujetos pueden ser individuales (cuentas) o grupos.
- Existe un nivel de configuración básico y otro avanzado que es más granular.
- Los permisos se puede heredar de un directorio a su contenido.
- La relación de herencia persiste a la creación, a diferencia de Unix



Windows File System - Herencia de permisos

Overview of Permission Inheritance

Permissions for Adam Barr

	Allow	Deny
Modify	☐	☐
Read & execute	☒	☐
List folder contents	☒	☐
Read	☒	☐
Write	☐	☐



Folder1



File1

Permission entries:

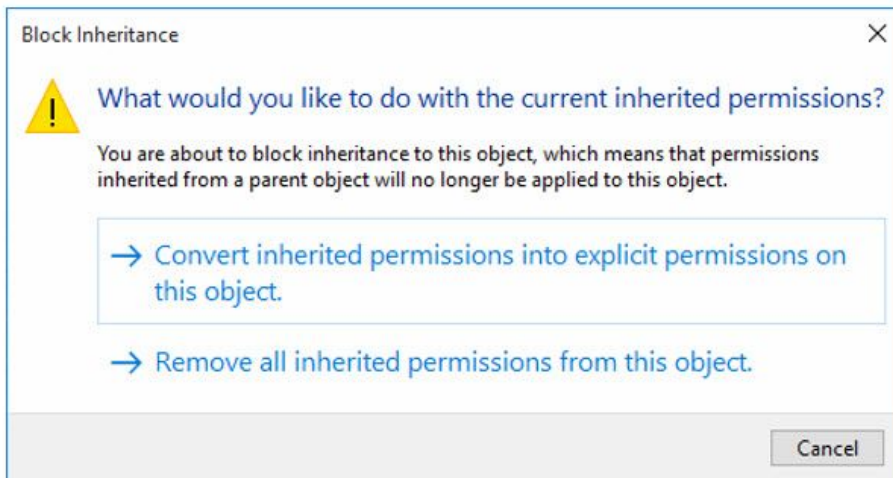
Type	Principal	Access	Inherited from
Allow	Adam Barr (Adam@adatum.com)	Read & execute	C:\Folder1\
Allow	Administrators (LON-CL2\Administrators)	Full control	C:\
Allow	SYSTEM	Full control	C:\
Allow	Users (LON-CL2\Users)	Read & execute	C:\
Allow	Authenticated Users	Modify	C:\



Folder1

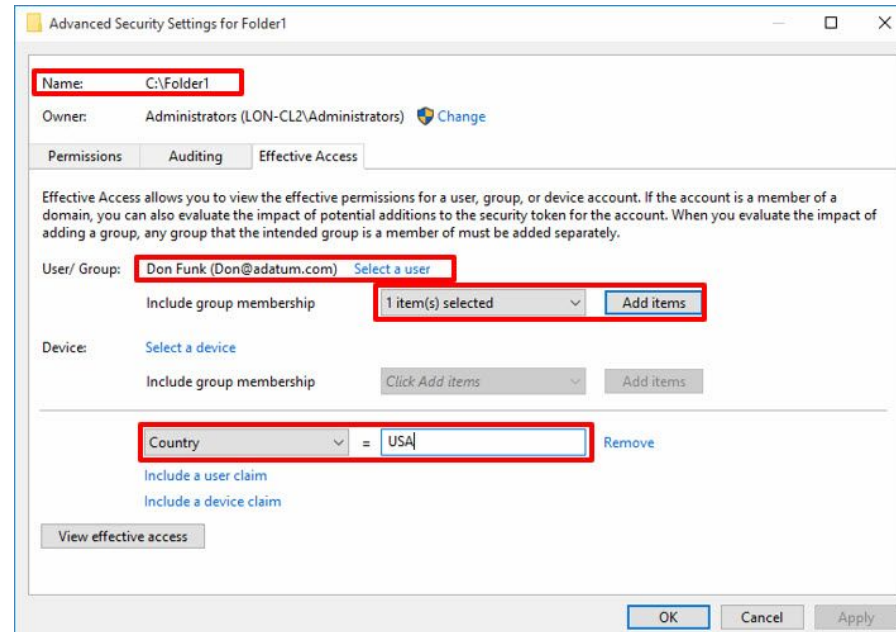


File2



Windows File System

- La cantidad de reglas superpuestas pueden ser muchas, de modo que Windows calcula los permisos efectivos que resultan de todas las reglas.
- Deny tiene precedencia sobre Allow
- Reglas locales tienen precedencia sobre reglas heredadas.
- Se puede usar un simulador para entender los permisos de un sujeto.



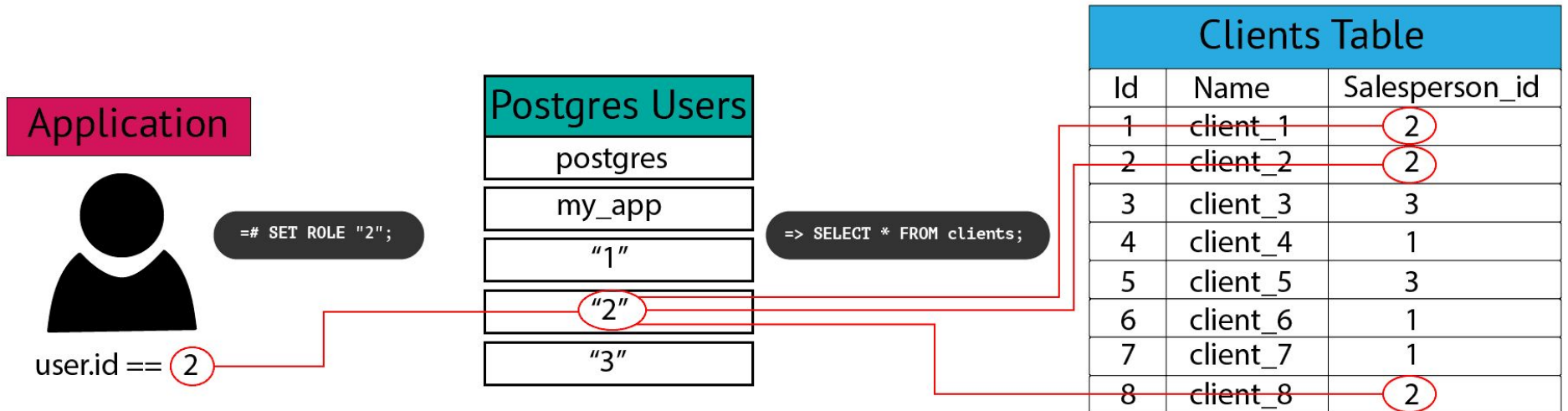
PostgreSQL - Roles

- Postgres utiliza un modelo de acceso basado en roles
- Un rol puede estar asociado a:
 - un usuario
 - un grupo de usuarios
 - otro rol
- Los roles poseen atributos que definen sus capacidades. En particular la propiedad LOGIN permite iniciar una sesión con la base de datos.
- Todos los objetos creados por un role, tienen como owner a ese rol (Otro caso de DAC). Sólo los superusers y el owner pueden acceder al objeto.

PostgreSQL - Policies

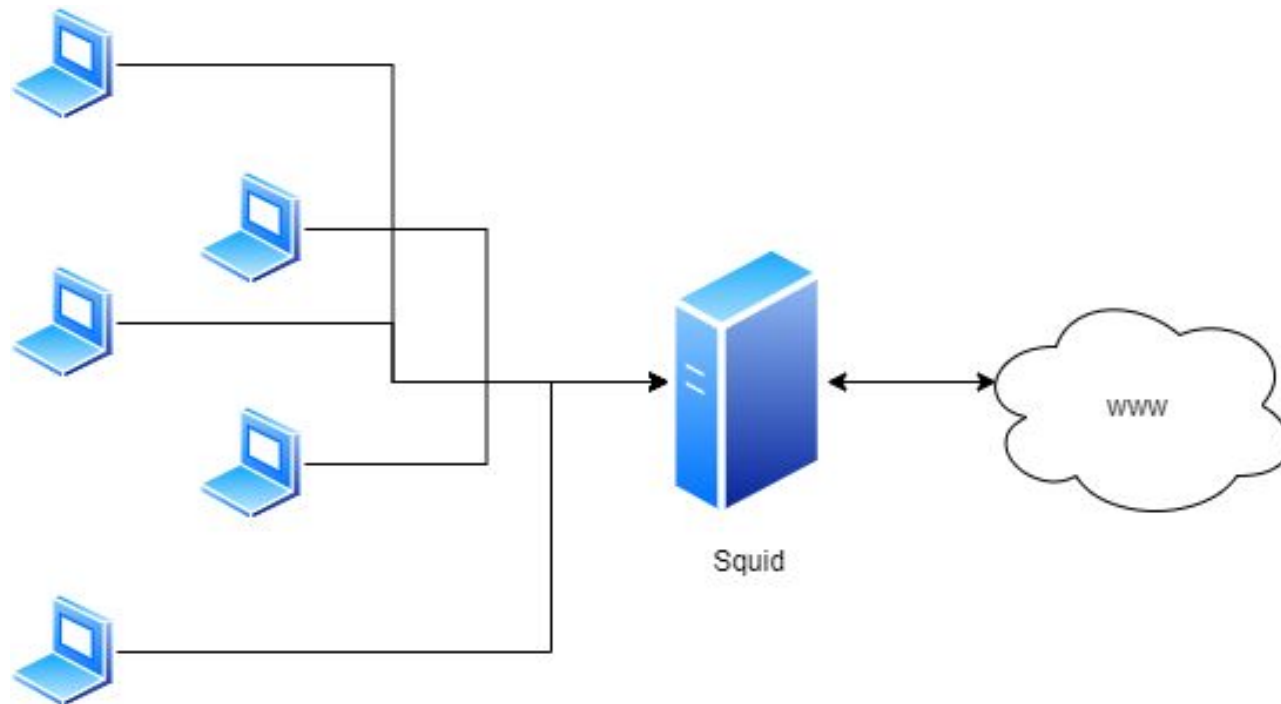
- Las políticas permiten la creación de políticas de acceso y modificación de registros dentro de las tablas
- Permite definir el need-to-know a nivel de registros

```
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;  
CREATE POLICY salesperson_clients ON clients USING (salesperson_id::text = current_user);
```



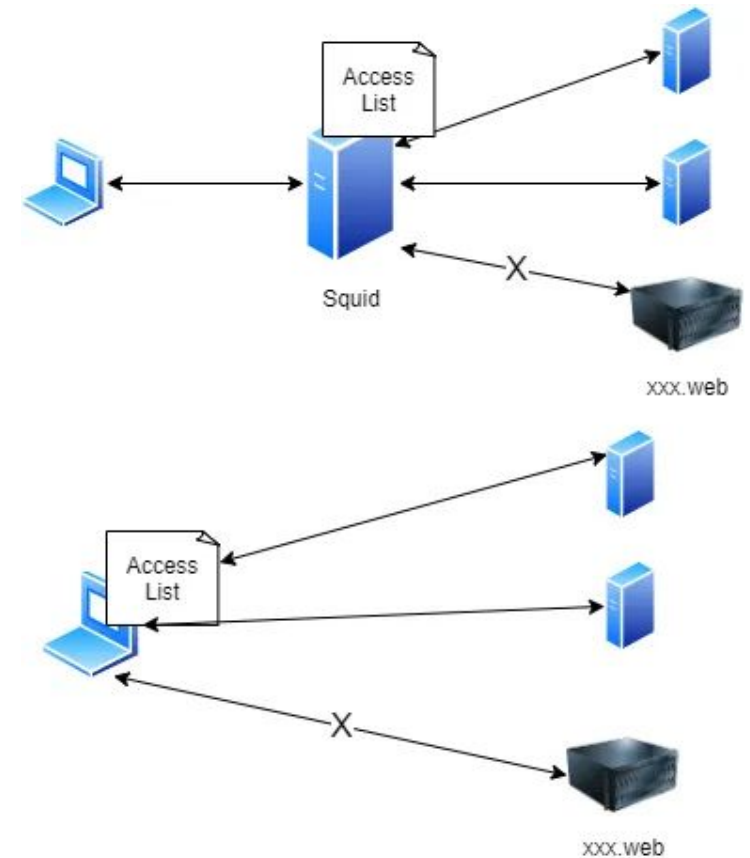
Squid Web Proxy

- Un web proxy es una herramienta que permite controlar por donde navegan los usuarios
- Además, cumple funciones de acelerar la navegación y reducir el consumo de ancho de banda.



Squid Web Proxy - ACL

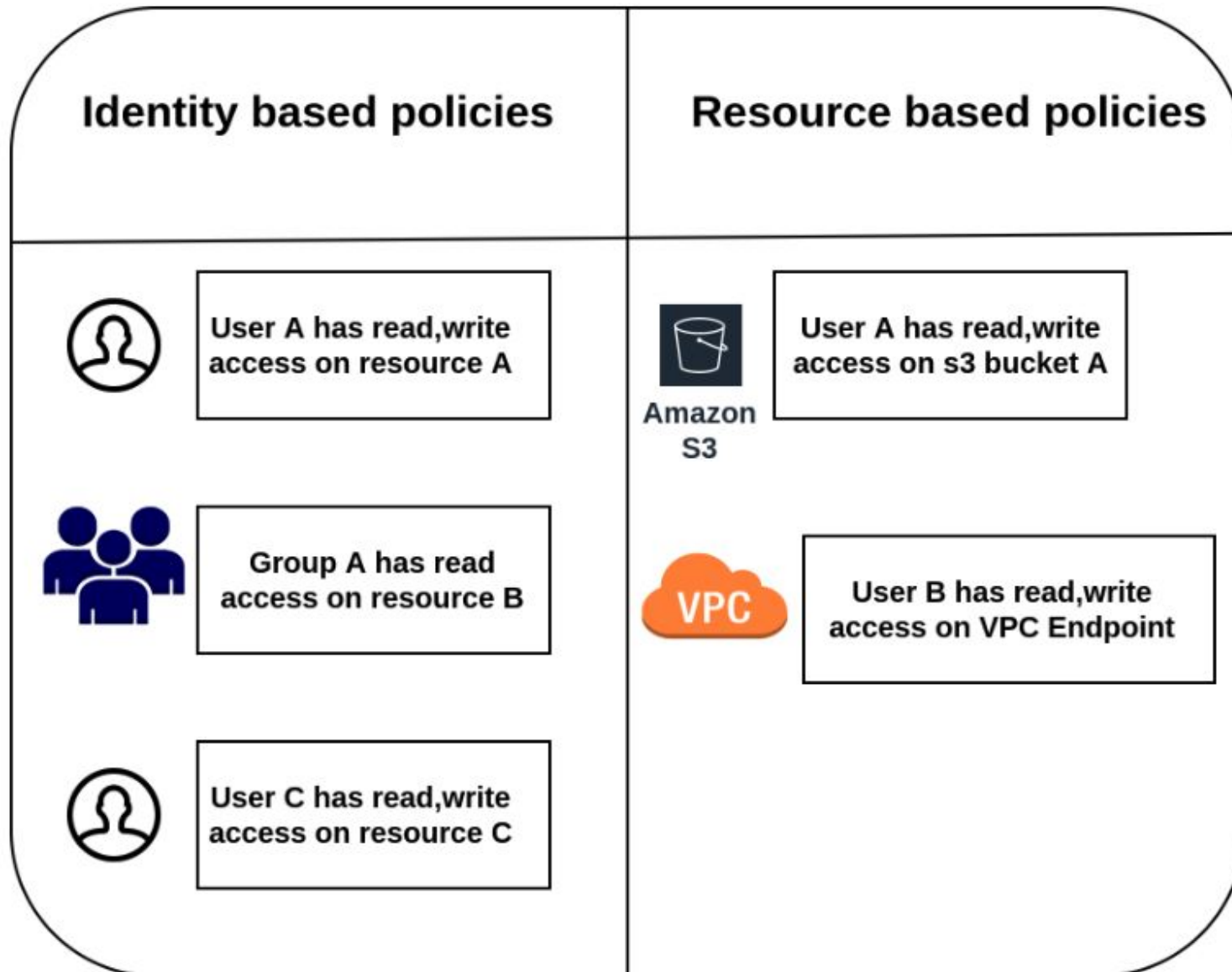
- Squid implementa un sistema de reglas (mal llamado ACL) MAC, donde se puede definir reglas para determinar qué sitios están permitidos o bloqueados
- En la facultad, los sitios de streaming están bloqueados usando este tipo de tecnología
- Utiliza reglas ordenadas que permiten filtrar por:
 - dominio del sitio, protocolo, URL, Headers del requerimiento
 - listas negras



Squid Web Proxy - Blacklist context rules

- Existen distintas iniciativas sin fines de lucro y otras comerciales para recolectar información de los ataques recientes.
- Información como la IP de origen del ataque, las metodologías usadas o el software (malware) usado quedan almacenadas en bases de datos.
- Estas listas se denominan blacklists, y pueden ser usadas por el Squid para bloquear a un usuario que trata de acceder a uno de estas IPs.

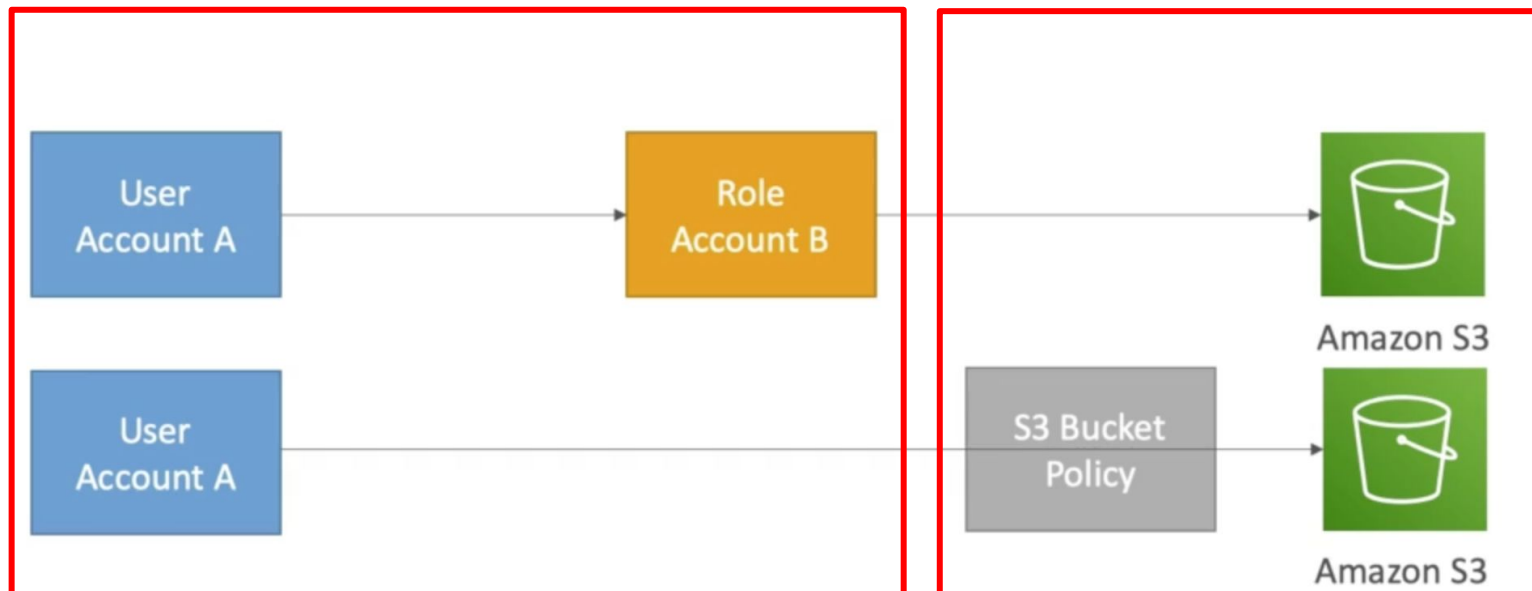
Ejemplo AWS



Ejemplo AWS

Policies asociadas
al Principal
(MAC)

Policies asociadas
al Recurso
(DAC)



API Access Control

- Un API gateway es un servicio que se despliega delante de las APIs para ofrecer una capa adicional de seguridad y funcionalidad
- Permite definir controles de acceso basados en reglas (MAC), que involucran parámetros como:
 - URL
 - Dominio
 - Headers
 - Method

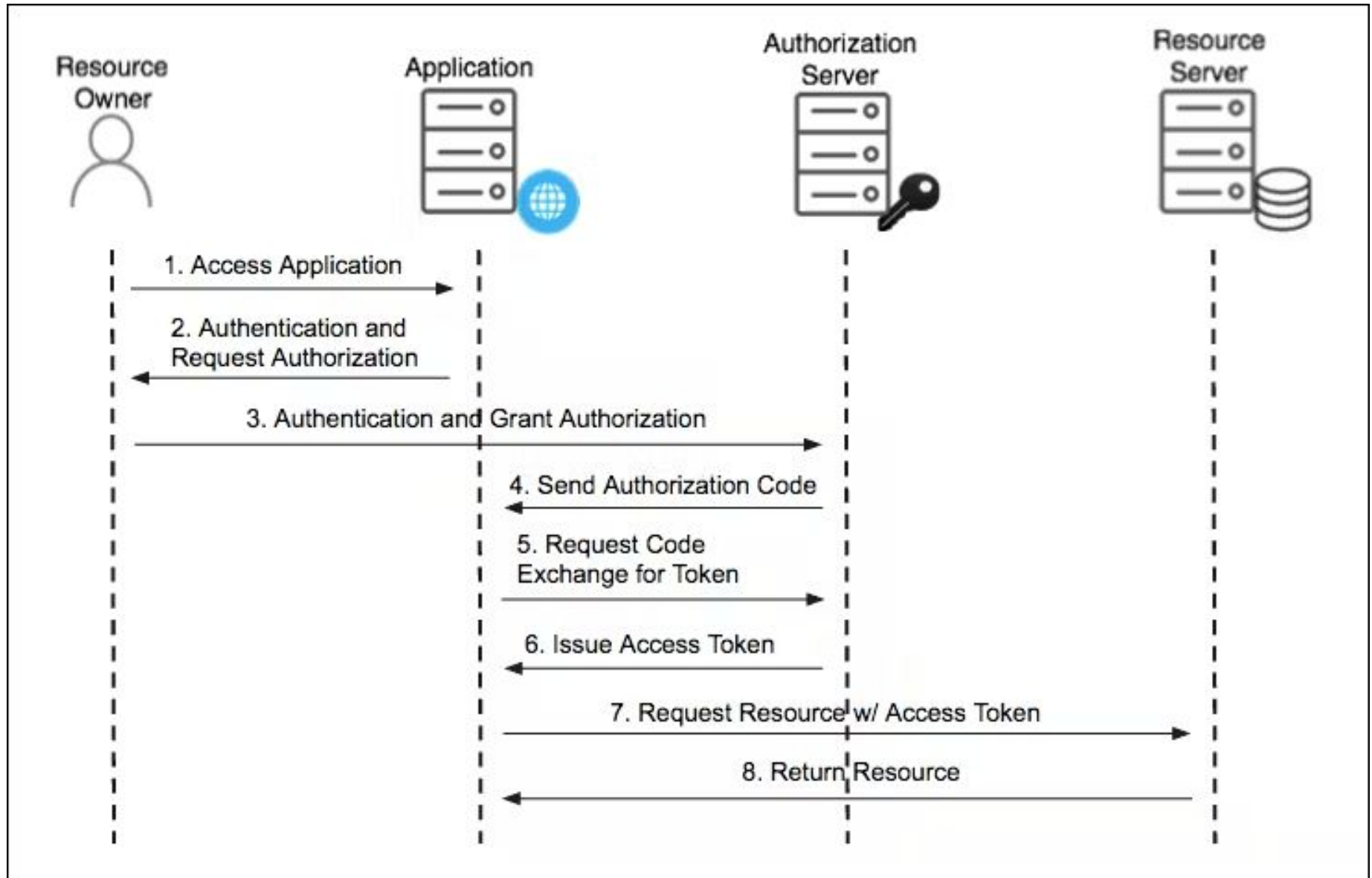
API Access Control por contexto

- Adicionalmente, puede definir algunas reglas basadas en contexto:
 - Cantidad de requerimientos por segundo (máxima)
 - Ancho de banda consumido (máximo)
- De las conexiones establecidas, mantiene un registro de consumo, y si se superan estos umbrales, se bloquean nuevas conexiones del mismo IP o SessionID.

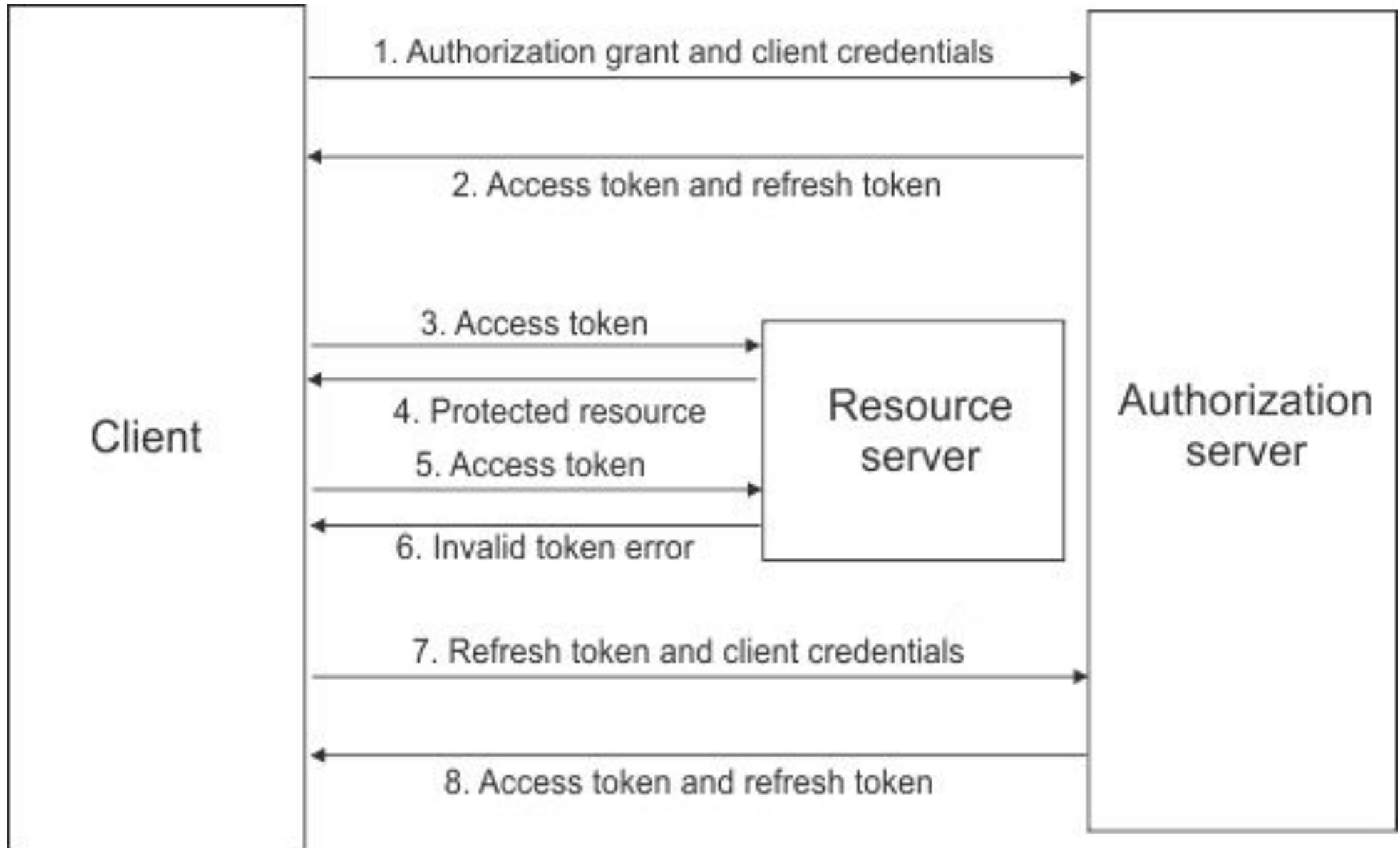
OAuth (Open Authorization)

- Estándar de autorización ampliamente utilizado para permitir que las aplicaciones accedan a recursos en una determinada plataforma en nombre del usuario sin que este tenga que compartir sus credenciales con la aplicación.
- OAuth funciona mediante la emisión de tokens de acceso temporales y específicos que permiten el acceso a los recursos del usuario.
- OAuth 2.0 es la versión más moderna y ampliamente implementada de este estándar y proporciona una flexibilidad mayor y seguridad adicional respecto a su predecesor, OAuth 1.0.

OAuth 2.0

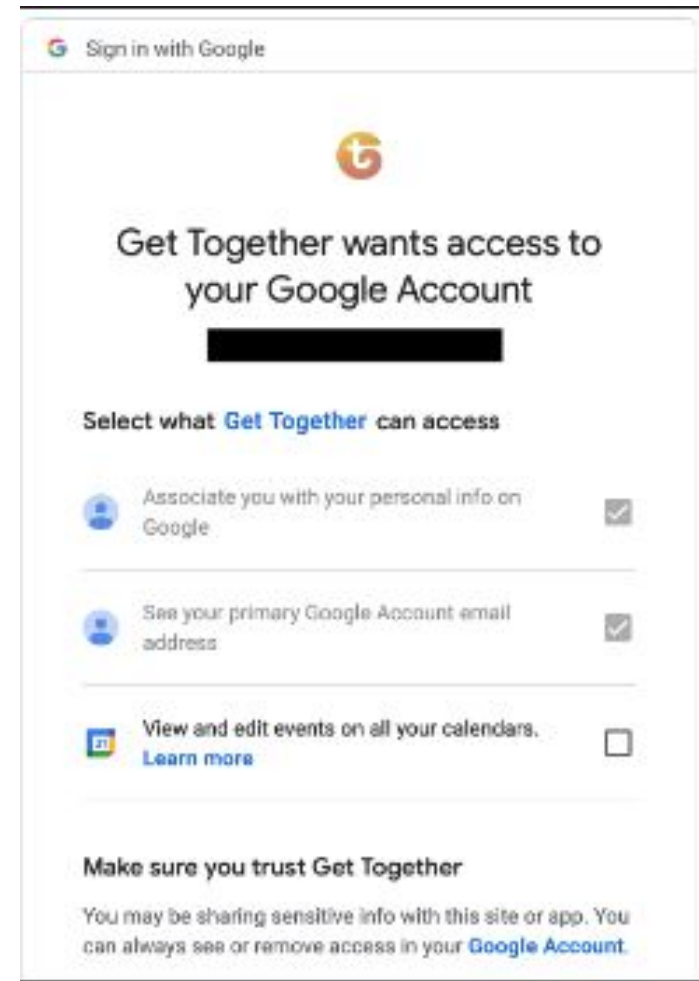


OAuth 2.0 (Token Refresh)



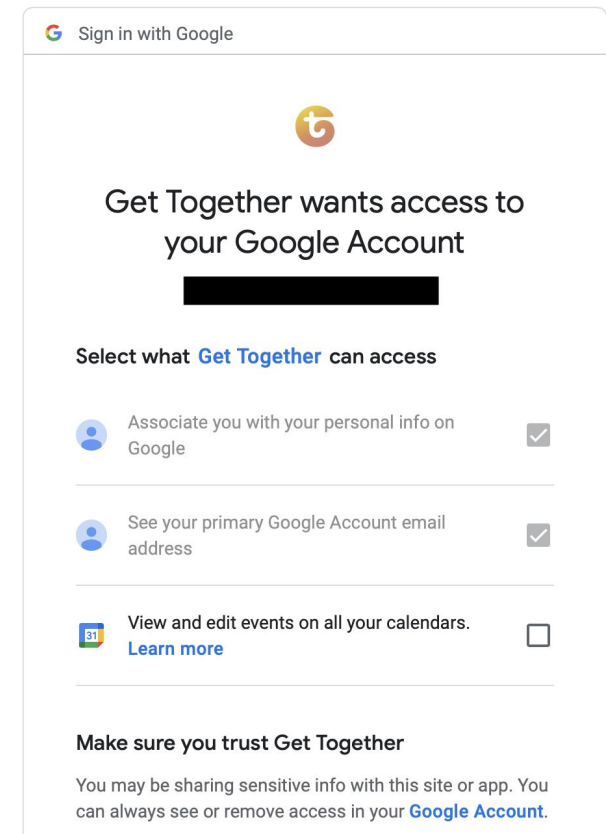
OAuth 2.0 (Scope)

- Scope (alcance) se refiere a los permisos específicos que una aplicación solicita al usuario para acceder a sus datos.
- Los scopes definen el nivel de acceso que la aplicación tendrá.
- Por ejemplo, una aplicación puede solicitar acceso solo a la dirección de correo electrónico del usuario, o puede pedir acceso a todos los contactos del usuario.



OAuth scope (DAC)

- Es un mecanismo de OAuth 2.0 para limitar el acceso de una aplicación a la cuenta de un usuario.
- Una aplicación puede solicitar uno o más scopes
- Esta información se presenta al usuario en la pantalla de consentimiento y el token de acceso emitido a la aplicación se limitará a los alcances otorgados.
- Cada servicio que recibe el token verifica si el mismo es válido y contiene su propio scope en la lista de scopes autorizados.



Lectura recomendada

Capítulo 4

Capítulo 5: 5.1-5.4

Capítulo 6: 6.1-6.2

Capítulo 7: 7.1

Capítulo 8: 8.1

Computer Security Art and Science

Matt Bishop